Python Rennes - mardi 17 mars 2026

Automatiser une release avec Github actions : montée de version, publication sur PyPI

Figure 1. Rediffusion vidéo : https://www.youtube.com/watch?v=KKBBOGeRPJg&t=578 (merci Alex pour la captation !)

Python Rennes - mardi 17 mars 2026 - Kanoma

Qui suis-je ?

- 💙💛 lead dev Python / Vuejs @ SeeYouSun 🌞⚡
- 🐈‍⬛ github.com/lucsorel (outils code-to-doc)
- 🐘 @lucsorelgiffo@floss.social

Publication de paquet sur PyPI

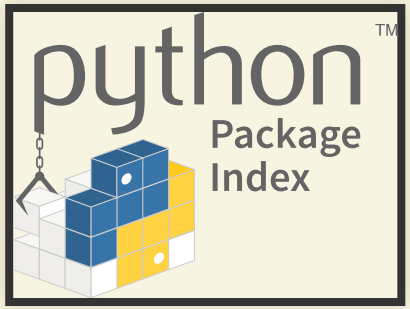


Figure 2. Le dépôt communautaire de paquets Python

- 765 531 projets (mars 2026)
- 8 262 044 versions
- ~5 milliards de téléchargements / jour
- dashboard public d'infrastructure : https://p.datadoghq.com/sb/7dc8b3250-85dcf667bd

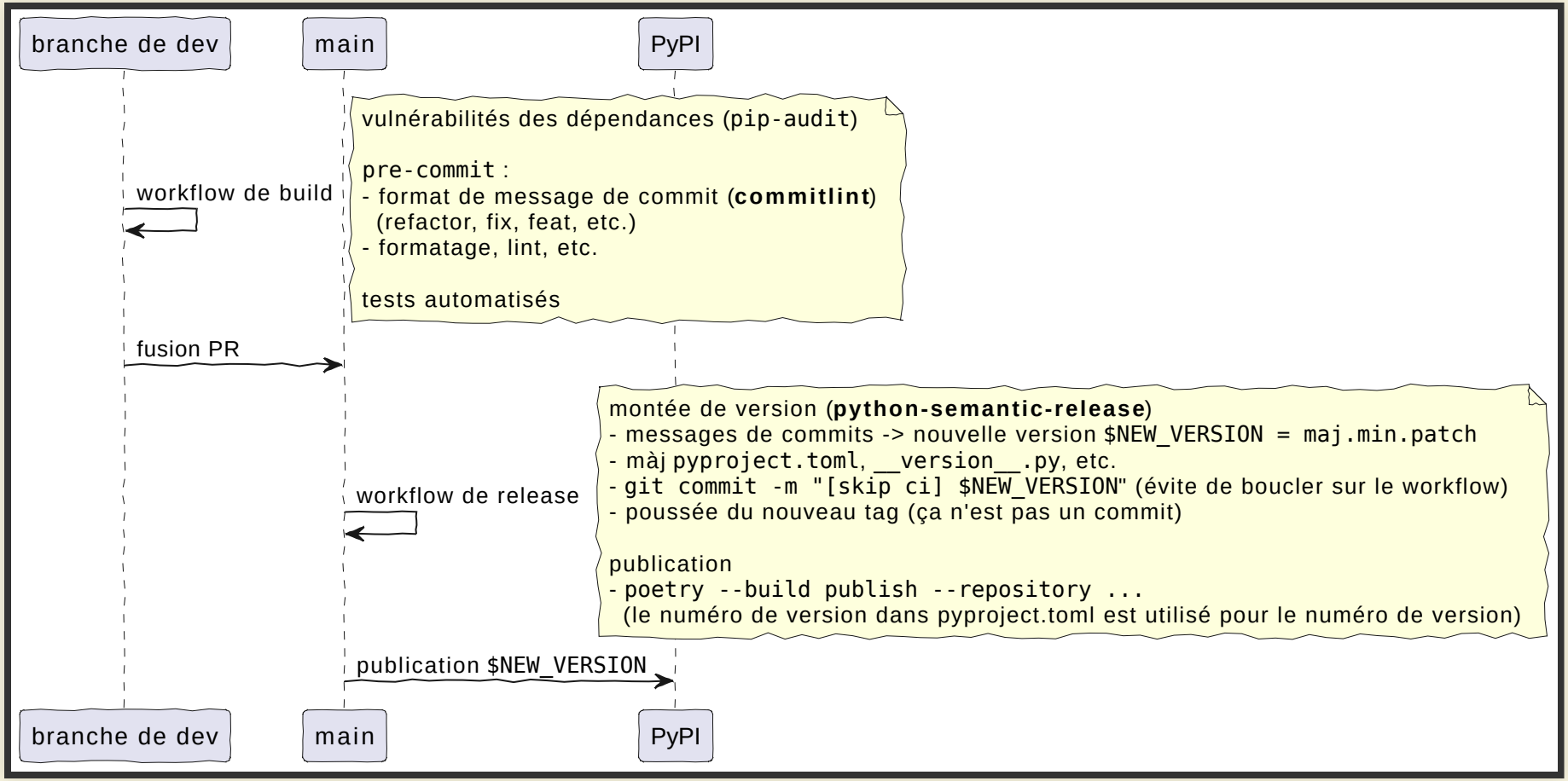
Souviens-toi… l'été dernier 😱

Figure 3. Retours d'expérience - intégration continue et déploiement de projets Python : https://www.youtube.com/watch?v=JdHFz67l-Ms (12 octobre 2023)

Hooks de pre-commit + workflow CI de qualité de code :

- conventional commits
- formatage et analyse statique de code
- tests automatisés

Un cliffhanger de 3 ans !



Dénouement

- il n’est plus possible de pousser un artefact sur PyPI depuis son poste avec ses credentials
- la majorité des corruptions de paquets ont été faites via des credentials volés
- encouragement du mécanisme "trusted publishing"

Sources :

- https://bernat.tech/posts/securing-python-supply-chain/
- https://blog.pypi.org/tags/#tag:security

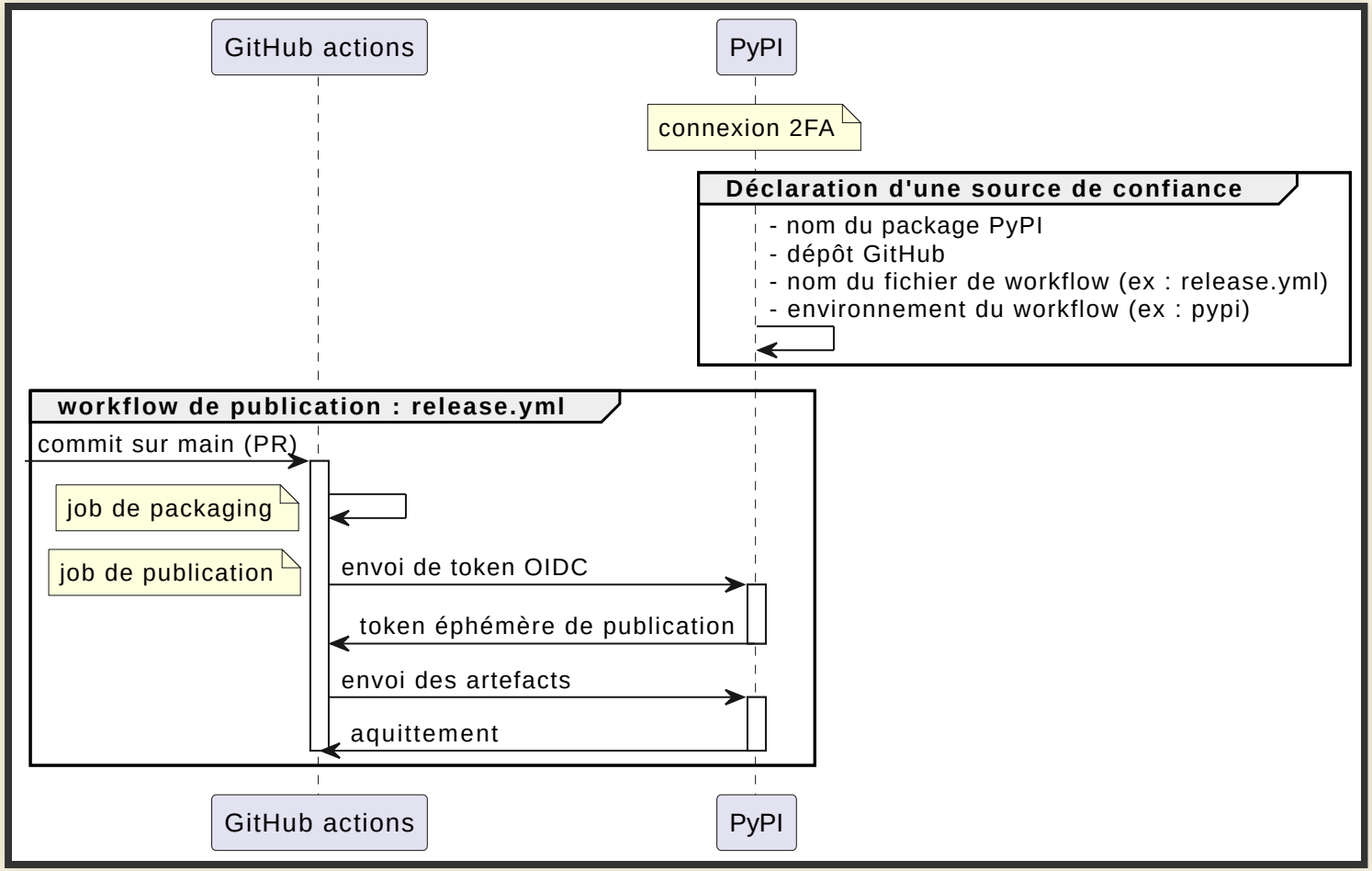
Trusted publishing : publication de confiance

- délégation de la publication à un processus de CI/CD
  - Github
  - Gitlab
  - ActiveState
  - Google
- tokens à courte durée de vie
- protocole OpenID Connect (OIDC), basé sur oAuth2.0

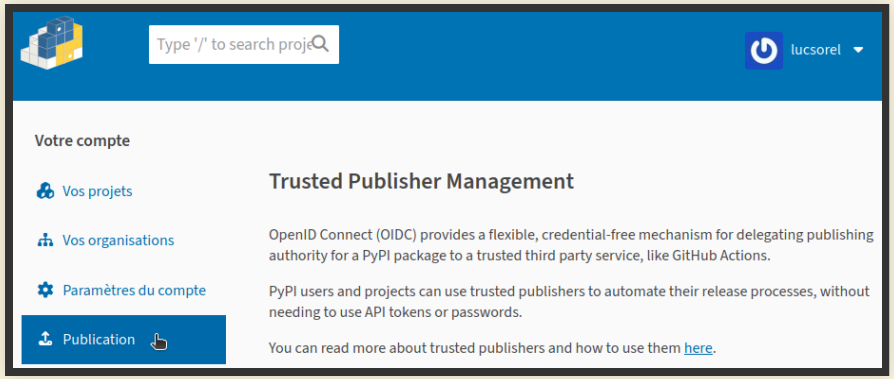
Sources :

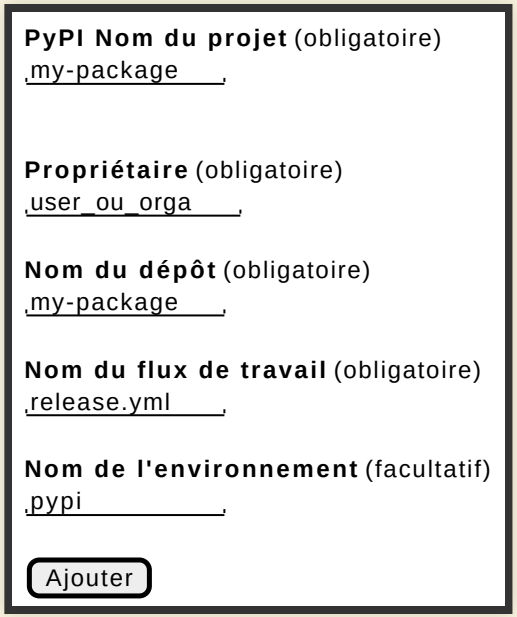
- https://docs.pypi.org/trusted-publishers/
- https://openid.net/developers/how-connect-works/

Trusted publishing : mécanisme général



Déclarer une source de publication de confiance





Mène à pypi.org/manage/account/publishing/

- dépot : `github.com/{user_ou_org}/{my-package}`
- workflow : `.github/workflows/release.yml`

(voir la documentation sur les environnements GitHub)

Workflows de publication 🏊

Plein de déclencheurs possibles (voir docs.github.com/…/events-that-trigger-workflows) :

- à la pose d'un tag de release (voir michelcaradec/horsebox)

```
on:
  release:
    types: [published]
```

- à la fusion d'une PR sur main

```
on:
  pull_request:
    branches: [ "main" ]
```

- au commit sur main

```
on:
  push:
    branches: [ "main" ]
```

Packager et pousser les artefacts 🏊🏊

- avec build et twine, flit (realpython.com/pypi-publish-python-package)
- uv build, uv publish (docs.astral.sh/uv/guides/package)
- poetry build, poetry publish (python-poetry.org/docs/libraries#publishing-to-pypi)
- GitHub action pypa/gh-action-pypi-publish@release/v1
- tutoriel realpython.com/pypi-publish-python-package
- voir aussi packaging.python.org/…/publishing-package-distribution-releases-using-github-actions-ci-cd-workflows

🤔 Mais en fait... 🏊🏊🏊

Qui dit publication, dit versionnement !

- semantic versioning (semver.org) : majeur.mineur.patch
- calendar versioning (calver.org) : YY.MM, YY.MM.DD, YY.mineur.micro

Versionnement automatisé avec python-semantic-release (PSR)

Projet : python-semantic-release.readthedocs.io

- calcul du prochain numéro de version via les messages de commits
- génération (initiale / incrémentale) d'un changelog
- insertion de la version dans la base de code (`pyproject.toml`, `__init__:__version__`, etc.)
- pose d'un tag de version
- packaging du projet
- publication (modifs, tags, artefacts) sur le repo

Proposition d'une configuration de CI/CD

- `code-quality.yml` : analyse statique de code, tests automatisés
- `release.yml` : sur `main`, quand `code-quality.yml` se passe bien
  - calcul du numéro de version
  - si nouvelle version :
    - modifications du projet (tag, changelog.md, pyproject.toml, uv.lock)
    - publication sur le repo des modifications
    - publication sur PyPI

Mise en oeuvre sur un projet utilisant uv

```
uv add --group package python-semantic-release
```

```
[dependency-groups]
dev = [
  ...
]
# pour les hooks de pre-commit
lint = ["pre-commit>=4.5.1"]
# pour le packaging
package = [
  "python-semantic-release>=10.5.3",
]

# les dépendances du groupe "package" ne seront pas installées par défaut
[tool.uv]
default-groups = ["dev", "lint"]
```

Configuration de PSR

```
# installation locale de python-semantic-release
uv sync --group package
# initialisation d'un fichier de configuration
uv run semantic-release generate-config > releaserc.toml
```

- semantic_release
  - version_toml : localisation de la version du paquet dans `pyproject.toml`
  - version_variables : localisations dans le code source
  - tag_format : "v{version}" → "{version}"
- semantic_release.branches.main
  - match : branches impliquant une montée de version
- semantic_release.changelog
  - mode : "init" / "update"
- semantic_release.commit_parser_options : conventional commits

Estimer le prochain numéro de version en local

```
[semantic_release.branches.main]
# activer temporairement la branche git courante
match = "(main|ma-branche)"
```

```
uv run semantic-release -c releaserc.toml -vv --noop version --print
```

Introduction aux conventional commits

Formalisme des messages de commit : https://www.conventionalcommits.org/en/v1.0.0/

Format de la 1re ligne ("l'entête") :

```
<type>(<scope>)!: <short summary>
 │       │       │    └─ *résumé* au présent, sans majuscule initiale, ni point final
 │       │       │
 │       │       └─ (facultatif) "!" force l'incrément du n° de version majeur
 │       │
 │       └─ (facultatif) *contexte métier* concerné par le commit : le·s nom·s du
 │           module Python, du package concerné, ou périmètre fonctionnel
 │
 └─ *type* du commit : build|ci|chore|docs|feat|fix|perf|refactor|style|test
```

Articulation avec le versionnement sémantique :

- "…!:" : incrément majeur
- `feat` : incrément mineur
- `fix`, `perf` : incrément patch
- autres types : modifications non fonctionnelles → pas d'incrément

Workflow release.yml - points d'attention 1/4

En détails : github.com/lucsorel/py2puml/blob/main/.github/workflows/release.yml

👀 "on" : `release` déclenchée après un workflow `code-quality` sur `main`

```yaml
name: release

on:
  workflow_run:
    branches: [main]
    workflows: ["code-quality"]
    types:
      - completed
```

👀 attention aux permissions read / write des différents jobs

```yaml
# lecture seule par défault
permissions:
  contents: read
# ...
jobs:
  release:
    # ... écriture dans le repo quand nécessaire
    permissions:
      contents: write
    steps:
      # ...
      - id: release
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          # ...
          uv run semantic-release -c releaserc.toml publish
```

Points d'attention 2/4

👀 forcer le checkout du code au commit et sur la branche qui a lancé le workflow

```yaml
jobs:
  package:
    # ...
    steps:
      - uses: actions/checkout@v5
        with:
          fetch-depth: 0
          ref: ${{ github.sha }}

      - run: git checkout -B ${{ github.ref_name }}
```

Points d'attention 3/4

👀 articulation conditionnelle des jobs :

- une étape de job peut écrire une valeur de sortie dans `$GITHUB_OUTPUT`
- un job peut exporter des valeurs dans `outputs`
- l'exécution d'un job peut être conditionnée

```yaml
jobs:
  package:
    steps:
      - id: version
        # c'est semantic-release qui exporte des valeurs associées à l'étape "version"
        run: uv run semantic-release -c releaserc.toml -v version --skip-build --no-commit --no-tag --no-changelog

    # sorties associées au job "package"
    outputs:
      new-release-detected: ${{ steps.version.outputs.released }}
      # ...

  release:
    needs:
      - package
    if: ${{ needs.package.outputs.new-release-detected == 'true' }}
```

(voir docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/pass-job-outputs)

Points d'attention 4/4

👀 sécurisation et configuration de la publication sur PyPI :

```yaml
jobs:
  deploy:
    if: ${{ needs.release.outputs.released == 'true' && github.repository == 'lucsorel/py2puml' }}
    needs:
      - release

    environment:
      name: pypi
      url: https://pypi.org/project/py2puml/

    permissions:
      contents: read
      id-token: write

    steps:
      # gère la demande de token éphémère et la publication de confiance sur PyPI
      - uses: pypa/gh-action-pypi-publish@release/v1
        with:
          packages-dir: dist
          print-hash: true
          verbose: true
```

Retours d'expérience

- le workflow de release 🥵 :
  - une architecture en soi·e 🤭
  - décisions d'articulation des outils
  - le tester sur test.pypi.org
- responsabilité des commits (contenu, message)
  - type → version
  - message → changelog
  - (se) former aux www.conventionalcommits.org

Garde-fous pour les commits

- .pre-commit-config.yaml

```yaml
- repo: https://github.com/alessandrojcm/commitlint-pre-commit-hook
  rev: v9.24.0
  hooks:
  - id: commitlint
    stages: [commit-msg]
    additional_dependencies: ['@commitlint/config-conventional']
```

- commitlint.config.js

```js
module.exports = {
    extends: [
        '@commitlint/config-conventional'
    ],
    rules: {
        // commit message 1st line and body of 150 characters max
        'header-max-length': [2, 'always', 150],
        'body-max-line-length': [2, 'always', 150],
    }
}
```

Accompagner la saisie des messages de commit

Extension codium / vsCode git-commit-plugin

`.vscode/settings.json`

```
{
  "GitCommitPlugin.MaxSubjectCharacters": 150,
  "GitCommitPlugin.ShowEmoji": false
}
```

Git power

Privilégier des pratiques git produisant un historique linéaire de commits pour faciliter le calcul de numéros de version.

```
# remanier l'historique des commits
git rebase -i {commit-hash}

# comportement au rebasage pour un historique linéaire
git config pull.rebase true
```

- préférer les rebasages de feature branches plutôt que des merges de la branche principale
- projet perso seulement pour l'instant
- accompagner l'équipe dans ses pratiques git

Merci ! 🙏

Des questions ?

Diaporama à retrouver sur github.com/lucsorel/conferences/tree/main/python-rennes-2026.03.17-ci-cd-projets-python-2